

Quantification of Cost-Resistance Tradeoff in Password Hashing Algorithms

Christoffer Larsen

Integrated Masters 3.year

UiT The Arctic University of Norway

Tromsø, Norway

chlar7639@uit.no

chlar7639@github

Deividas Kalnikas

Department of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

dekal0739@uit.no

ImmortalStogh@github

I. INTRODUCTION

Password security still largely comes down to one question, how the system stores and verifies authentication keys. Storing passwords in plaintext or simple hashing functions leaves the system exposed because of low entropy and randomness. Today ASIC hardware and GPU can compute billions of such hashes in seconds, making brute-force and dictionary attacks easy for password cracking. To address this, password specific hashing algorithms were created like SHA-256 and BLAKE3 to be slow and resource intensive. Making brute-force attacks computationally expensive.

This project investigates and compares the most widely used password hashing algorithms including SHA-256, BLAKE3, Argon2id, scrypt, bcrypt, and PBKDF2 with a focus on their cost parameters and how those parameters influence both system performance and resistance to attack.

A. Motivation and goal

The primary challenge in password hashing is compensating two competing demands. On one hand, the hashing process must remain fast enough that users experience acceptable login latency. On the other hand, it must be slow and resource intensive enough that attackers attempting to crack stolen password hashes typically using GPU accelerated brute force or dictionary attacks face a significant computational barrier. Every Resource parameter (such as memory usage, latency, iteration count and parallelism) sit somewhere on this trade off curve, and choosing the wrong configurations can either frustrate users or leave the system vulnerable.

This matters because password breaches remain one of the most common and damaging security incidents, and the choice of hashing algorithm and configurations is often the last line of defense once a database has been compromised. Despite this many still use outdated hashing algorithms. This project aims to provide practical, evidence based on selecting and tuning a hashing algorithms that offers the best resistance against attackers within an acceptable latency budget.

B. Research questions

Several research questions are put into place:

- Question 1: How will each algorithm's parameters affect the authentication latency?
- Question 2: How will the parameter cost affect the attacker's cracking throughput (guess/sec) on GPU hardware?
- Question 3: Given an "acceptable" latency budget, which combination of parameter and algorithm will provide the best resistance against a given time budget?

C. Hypotheses

The hypotheses set in place are:

- Hypothesis 1: Memory-hard algorithms will have a much steeper GPU-cracking slowdown per unit of legit-login latency cost than CPU-hard algorithms, because GPUs excel at parallel computing, but at memory limited.
- Hypothesis 2: CPU-hard algorithms will be cracked at a meaningfully higher throughput than the GPU based, even at equivalent time limits, thus exposing memory-hardness as the dominant factor rather than time cost.

II. RELATED WORK

Research on hashing methods has established that compared PBKDF2, Bcrypt, Scrypt in terms of implementation and performance, evaluation factors like execution time and resistance to brute force/hardware attacks [2]. The study was limited to a specific set of parameters/hardware and did not address real world deployment.

Biryukov, Dinu & Khovratovich [1] introduced Argon2, the winner of the password hashing competition, as a new generation of memory hard functions designed to resist GPU/ASIC cracking attacks. There are two variants (Argon2d and Argon2i) balancing resistance to GPU attacks against resistance to side-channel attacks, along with tunable cost parameters. Strong theoretical and competition backed grounding, but as a foundational paper, it focused on a more broader view rather than empirical comparison against other hashing algorithms in real world deployment context.

Pandya [3] evaluated the performance of three widely used hashing algorithms, BLAKE3, SHA-256 and SHA-512 in context of blockchain application, benchmarking hashrate and latency across multiple platforms. It was found that BLAKE3 generally outperforms both SHA-256 and SHA-512 in throughput and latency, though the performance advantage varies depending on the platform and data size. The study offers useful cross platform benchmarking but focuses generally on blockchain hashing functions rather than password hashing algorithms specifically, and doesn't address resistance to brute force attacks.

Sprengers sets up the and provides the numerical data for the "GPUs broke the old assumptions" narrative, justifying why this is a real problem [5], while Prokarovsky et al. established a good methodology precedent using the same tools and the comparison structures as planned for this project [4].

III. SYSTEM

The system will consist of a *sweep controller* that will initiate, define parameters and iterate runs. The hashing algorithm hashes the password with it's set parameters and records the latency, after that it is given to the attacker pipeline and attempted to be cracked within a given time. Regardless of the results, the metrics are recorded and tagged.

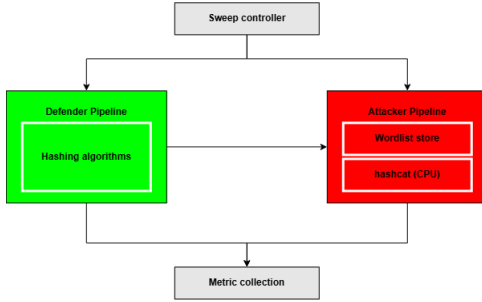


Fig. 1. Component diagram showcasing the system for the metric

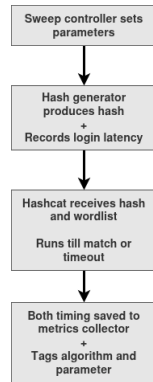


Fig. 2. Sequence diagram showcasing the sequence in which the metric will be collected in.

IV. HARDWARE ENVIRONMENT

Benchmark machines representing a defender and attacker roles, will be used to measure our research questions. Ideally a machine with a multi core CPU, to simulate realistic behavior from server when it comes to hashing, memory distribution and parallelism. The project does not require any external network or production access, as testing is self contained. The experiments will run in an isolated, offline test environment to avoid any risk to live systems or real credential data.

The machine used for experimenting has Intel i5-8300H CPU with 2.3 GHz as its clock speed as base and 4 GHz as max, housing 4 cores with a total of 8 threads. Machine's GPU is an NVIDIA GeForce GTX 1060 Mobile version with 3072 MiB VRAM and 1280 CUDA cores. System's RAM capacity at 8 GB (7.6 GiB visible). There are no network isolation layers in play, the test machine runs at offline only.

V. SOFTWARE ENVIRONMENT

The test machines's OS configuration is Arch Linux, and both the attacker and the defender is set on the same machine with the same parameters. Present hashing algorithms will have their respective libraries: *SHA-256*, *PBKDF2*, and *scrypt* uses Python's built in *hashlib*, which wraps OpenSSL, *bcrypt* uses its own package from PyPI, *argon2id* uses *argon2-cffi*. These are widely used and maintained libraries, and would be better optimized than if written by hand. Attacker's side would consists of *hashcat*'s version 7.1.2 on the GPU via OpenCL, using its benchmark *-b* for throughput and mask attacks for certain validation runs.

A python script will loop over the algorithms, their cost settings, time each, call for hashcat and later write the result out. The logging format would mostly be compatible with the analysis and plotting tools available to python, *pandas* and *matplotlib*, set to CSV.

The system has no simulation tools. All measurements are taken on physical hardware already described.

VI. WORKLOADS

We evaluate five password hashing configurations spanning the range of cost structures in common use, from no cost parameter at all to fully memory-hard, multi parameter hashes. Since each algorithm has its own way of measuring cost, we can't tune a single shared 'cost' setting across all of them. Instead, we set separate parameters for each algorithm and make the comparisons fair afterward, during analysis.

For Scrypt and Argon2id, the block size and parallelism are held at defaults rather than then swept, since sweeping all axes simultaneously produces a combinatorial grid whose additional cells are largely uninformative for our resource

TABLE I
PARAMETERS VARIED FOR EACH HASHING ALGORITHM

Algorithm	Parameter(s) Varied	Range	Held Const.
SHA-256	— (no cost param.)	single iteration	—
Bcrypt	cost factor ($\log_2$ rounds)	4, 6, 8, 10, 12, 14	—
PBKDF2-SHA256	iteration count	$10^3, 10^4, 10^5, 3.1 \times 10^5, 6 \times 10^5$	PRF = SHA-256
Scrypt	N (CPU/mem cost, pow. of 2)	$2^{10}-2^{20}$	$r = 8, p = 1$
Argon2id	m_{cost} (KiB), t_{cost} (iter.)	$m_{\text{cost}}: 2^{12}-2^{18}; t_{\text{cost}}: 1-4$	$p = 1$

questions. The memory cost and CPU cost are the dominant levers determining both authentication latency and GPU resistance. We intentionally narrowed down what we tested. Argon2id has 4 settings you can tune, and testing every possible combination of them would produce way too many test cases to be practical. On top of that, most of those combinations aren’t ones anyone would actually use in a real system — so testing them all would mostly be wasted effort.

SHA-256 is included specifically as a negative control. It has no work factor parameter, so it contributes as a single reference point, rather than a swept curve, anchoring the “zero cost” extreme against which the tunable algorithms resistance gains can be measured.

For the first resource question, we require the modeling of a legitimate login path, which runs on defender controlled CPU hardware rather than attacker GPU hardware, and requires measuring the cost of a single hash operation rather than aggregate throughput. This tool isn’t a good fit here. Its speed numbers come from cracking many passwords in parallel using its own fast, custom-built code for each algorithm — not from a single password check using the standard library function an authentication server actually uses. Instead, we time how long a single login attempt takes calling the standard implementation of each algorithm once, at each setting in our test grid.

What we measure: How long it takes to hash one password of a fixed length, in real time. How we test it: We first run the hash a bunch of times without recording results, to “warm up” the system. Then we time a minimum number of actual runs for each setting. We report the median time as our main number, since a single run can randomly take much longer due to OS level noise, and we also report the p95/max times separately, to talk about those worst case slowdowns. We extract guesses/ sampled during the steady portion of the run, excluding the brief warm up window at start. For our memory hard algorithms, the throughput is additionally gated by available device memory, which limits the number

of parallel runs that fit in virtual memory at high cost. We track this explicitly rather than treating it as noise, since the throughput “cliff” at high memory cost, and any hard ceiling imposed by available resource, is itself relevant to our resource question and may constrain how far the upper end of our hashes parameter ranges can actually be tested. The output of algorithms an parameter settings will be guesses/sec, forming the start to our Q3.

For the second resource question, it requires measuring attacker side throughput on GPU hardware. We use a brute force attack against a target set of hashed rockyou passwords, rather than a wordlist based attack. The targets are real passwords from a past breach, so cracking success reflects passwords people actually choose, but the attack itself doesn’t look them up in a dictionary. Instead, it generates candidate guesses from a defined character set and checks each one against the hashes, until it either finds a match or time runs out. This measures raw guessing speed against realistic targets, rather than being limited by wordlist coverage or disk I/O.

Before timing anything, we throw away the first several runs. This stops one time costs from being mistaken for the algorithm’s actual cost. It matters most for cheap operations like SHA-256 or low-setting bcrypt/PBKDF2, where the real work only takes microseconds, fixed overhead would otherwise swamp the result.

GPU kernels take a moment to hit full speed after launching. So we measure guesses/sec only after that ramp-up window, not from the first few seconds — otherwise we’d underestimate throughput.

VII. DATASET(S)

Password samples for this study come from two sources. Real leaked passwords and a set of synthetic passwords generated for the purpose. For leaked passwords the study uses the RockYou dataset, originating from 2009. this is something of a standard in password security research and appears throughout the academic literature on password strength and cracking.

The synthetic passwords are generated with specific parameters such as length, character set and structure. This is intentional design choice that follows from the focus of the study. The research question concerns the behavior of the hashing schemes, including their resistance to cracking and their computational cost, and not the strength of the passwords themselves. if the password strength were left to vary freely, differences in cracking outcomes could not cleanly attributed to the hashing scheme, because they might just as easily result from some passwords being easier to guess than others. Fixing the generation parameters removes this confounding factor. Every scheme is then evaluated on identical, known inputs, so that differences in cracking time or success rate can be attributed to the hashing algorithm and its configuration rather than to the input. The controlled parameters also make the experiments reproducible, since the same password sets can be

regenerated and reused across schemes and test runs, and they allow the difficulty of the input to be set at known levels, for example by generating groups of passwords of a given length and character-class composition so that the hashing schemes can be compared at each level. Synthetic passwords also raise no ethical concerns, as they correspond to no real person or account.

The leaked password are more of an possible extension of the study, in the sense we want to extend the scope.

VIII. EVALUATION AND PERFORMANCE METRICS

The framework would give out a mean hash/sec, and the mean, benchmarked guess/sec metrics, followed after each configuration and each work. As a whole, the framework will be focused on two measurements at the end of its pipeline: the keyspace exhaustion and the attacker advantage ratio. Both take on RQ3 and will showcase the differences in HQ.

A. Keyspace exhaustion

Keyspace exhaustion refers to the amount of time it takes to exhaust a given keyspace for the given cracking/benchmarking tool. It's calculated by diving all possible combinations of a given keyspace with the speed of the tool, e.g. guess/sec. In return it gives the maximum time for the worst case scenario.

B. Attacker advantage ratio

This particular ratio focuses on setting the hashing speed of the scheme against the cracking speed of the benchmarking tool. The given answer displays the theoretical amount of hashed passwords possible per the amount of each guess. Since both quantities are throughput measurement taken independently of any particular password, the showcased the ratio therefore characterizes the algorithm and the hardware. Thus it can be potentially used a hypothetical ratio and the hypothetical baseline for other systems.

IX. EXPERIMENTAL PROTOCOLS

The protocol for each password fed into the scheme's are going to follow MFA-case floor, 8 character, randomly generated password. It will be set to calculate the hashing time 100 times to get a good mean, after warmup, with the possibility to increase the amount in case there's too much variety in the measured time.

A. Confounders and mitigation

- **Thermal throttling:** both the CPU and the GPU are laptop parts. GPU temperature is logged during every Hashcat run.
- **CPU frequency scaling:** the CPU governor is fixed for all latency runs and the machine is kept on mains power.
- **Background load:** no other user applications run during measurements, and no other process uses the GPU.
- **Driver and software versions:** kept constant throughout (NVIDIA driver 580.178.04, CUDA 13.0, Hashcat 7.1.2).
- **Memory contention:** defender and attacker measurements are never run at the same time.

X. TIMELINE

The setup and literature review will take 1-3 weeks. Baseline implementation will take 1-2 weeks. Benchmarking could will take approximately 4 weeks for latency and attacker throughput benchmarking. Then about 4 more weeks for cross analysis, report writing and presentation. These numbers can be changed to fit the agenda.

XI. DISCUSSION

Consumer grade GPUs used for testing may not reflect the capabilities of well funded attackers using clusters of high end GPUs showing the lower absolute guesses/sec, but the relative comparisons between algorithms still holds, which is what the hypotheses depend on.

Five algorithms * multiple parameters * two measurement dimensions(latency and throughput) is a substantial testing amount. There's a risk of insufficient time to complete full sweep for all algorithms. a fallback plan would be to prioritize some of the algorithms and change the amount of testing to fit the time limit.

Benchmark results can be sensitive to background system load, heat or driver versions. Running controlled tests, contained environment and repeating each measurement multiple times will help mitigate noisy or non reproducible results.

Hashcat's brute force mode, known as a mask attack, is "informed" because you tell it the structure of the password space rather than the password itself. Using placeholders like ?l (lowercase), ?u (uppercase), ?d (digit), and ?s (special character), you build a template such as ?u?l?l?l?d?d?s that specifies, in correct positional order, which character class applies at each position for example, an uppercase letter first, four lowercase letters next, three digits, then a special character. Hashcat then tries every possible combination that fits this shape, without knowing the actual characters in advance.

This means that we are not using pure brute force attacks, but rather from a mask of different letters,digits and special signs. The trade off is that accuracy in the assumed structure directly determines success. A well informed mask can crack a password quickly, but if the real password doesn't match the specified pattern, Hashcat will never find it no matter how long it runs, which is why mask attacks differ from dictionary attacks (using wordlists) or hybrid attacks (combining wordlists with masks).

Another concern is that Argon2id is a memory hard scheme witch means it is limited by the hardware when it comes to testing. This will be clearly established within the exepriments.

REFERENCES

- [1] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. “Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications”. In: *2016 IEEE European Symposium on Security and Privacy (EuroSP)*. 2016, pp. 292–302. DOI: 10.1109/EuroSP.2016.31.
- [2] Levent Ertaul, Manpreet Kaur, and Venkata A. K. R. Gudise. “Implementation and Performance Analysis of PBKDF2, Bcrypt, Scrypt Algorithms”. English. In: *Proceedings of the International Conference on Wireless Networks (ICWN)*. 2016, pp. 66–72. URL: <https://www.proquest.com/conference-papers-proceedings/implementation-performance-analysis-pbkdf2-bcrypt/docview/1817061029/se-2>.
- [3] Marut Pandya. *Performance Evaluation of Hashing Algorithms on Commodity Hardware*. 2024. arXiv: 2407.08284 [cs.CR]. URL: <https://arxiv.org/abs/2407.08284>.
- [4] Stefan Pocarovsky et al. “Computational Efficiency of Brute-Force Attacks on Hashing Algorithms”. In: *Software Engineering: Emerging Trends and Practices in System Development*. Ed. by Radek Silhavy and Petr Silhavy. Cham: Springer Nature Switzerland, 2026, pp. 402–413. ISBN: 978-3-032-00236-5.
- [5] Martijn Sprengers. “GPU-based password cracking”. In: *On the Security of Password Hashing Schemes regarding Advances in Graphics Processing Units, Radboud University Nijmegen* (2011).